

COS 214 Practical 4: 08 September 2026

Bianca Kritzman u25144465

Christian Nunes u25138309

Tamima Alam Touhid u24691284

System description:

The system revolves around an emergency response. There is a hierarchical emergency response operation managing responders and incidents, supporting flexible traversal, dynamic modification, and runtime responsibilities

Traversal-Modification Policy:

Traversal policy: Snapshot (for reports) and Live (for real-time dispatching)

GitHub link: https://github.com/BiancaKritzman/Prac4_COS214.git

GoF participant mapping:

Client - main

Component - ResponseComponent

Composite / ConcreteAggregate - ResponseTeam

Leaf / ConcreteComponent - Responder

Iterator - ResponseMovement

ConcreteIterators - LiveResponseMovement, SnapshotResponseMovement

Context - Responder

State - ResponderState

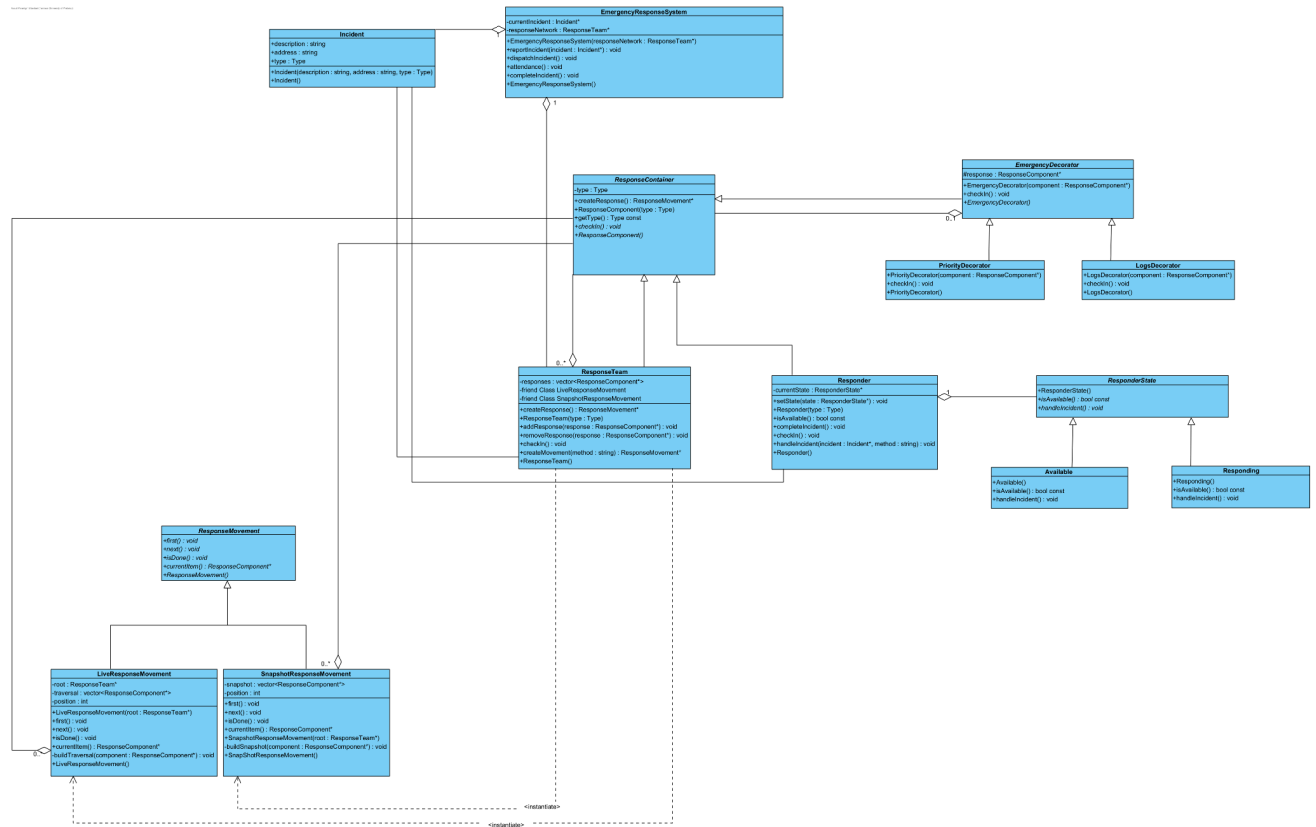
ConcreteStates - Available, Responding

Decorator - EmergencyDecorator

ConcreteDecorators - PriorityDecorator, LogsDecorator

Application/controller class - EmergencyResponseSystem

Domain object - Incident



TASK 3

Our Emergency Response System demonstrates two runtime scenarios where the Composite, Iterator, State and Decorator patterns work together.

In the first scenario, a medical emergency is reported and responders from the nested **ResponseTeam** hierarchy are used to handle the incident. Responders begin in the **Available** state and change to **Responding** when assigned to an incident. Once the incident is completed, they return to Available. A decorated responder also participates using stacked logging and **PriorityDecorator** behaviour while still performing normal responder operations.

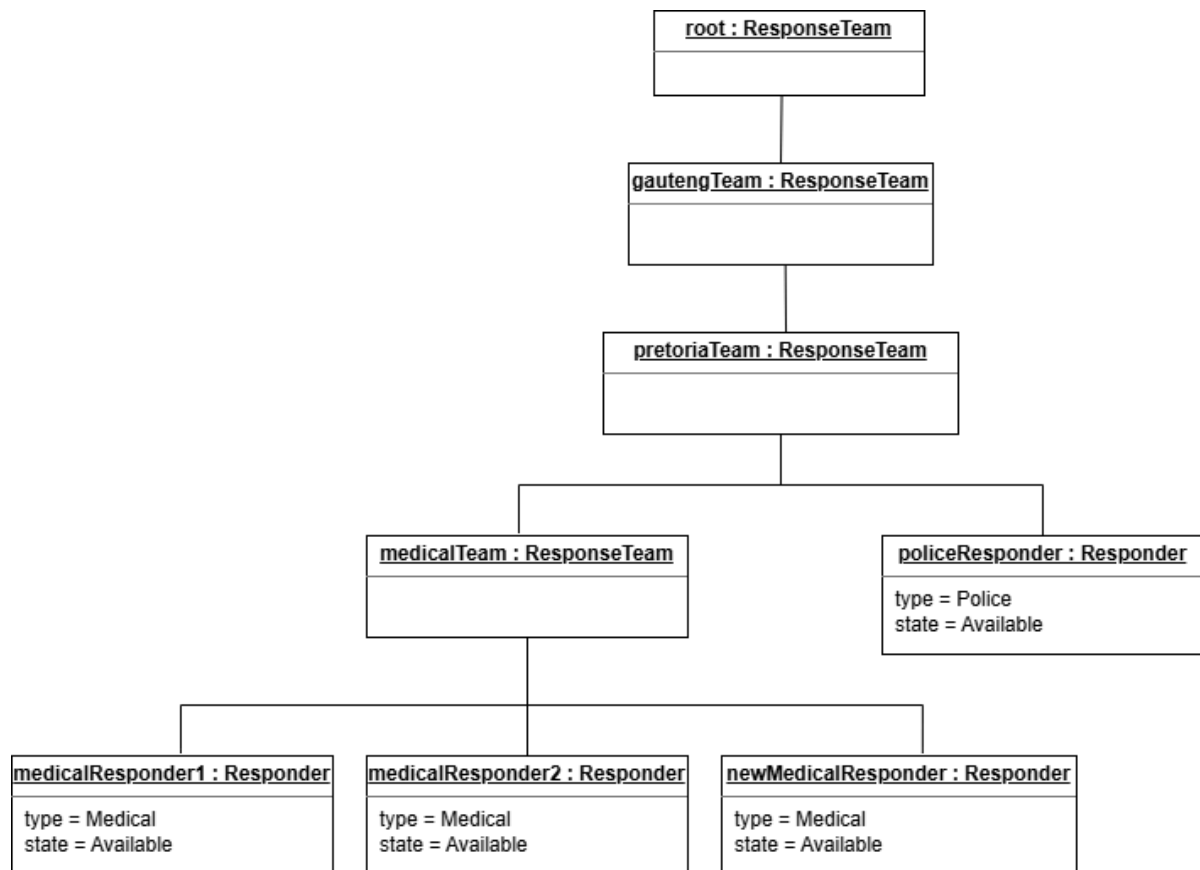
In the second scenario, the hierarchy is changed at runtime. A snapshot traversal is created before a new medical responder is added. The existing snapshot keeps the structure as it was when created, so the new responder is not included. The live traversal uses the current hierarchy and therefore includes the newly added responder.

Our traversal policy deliberately supports both behaviours: **snapshot traversal remains unchanged after structural changes, while live traversal reflects runtime additions**. This provides predictable traversal while still allowing the system to change dynamically.

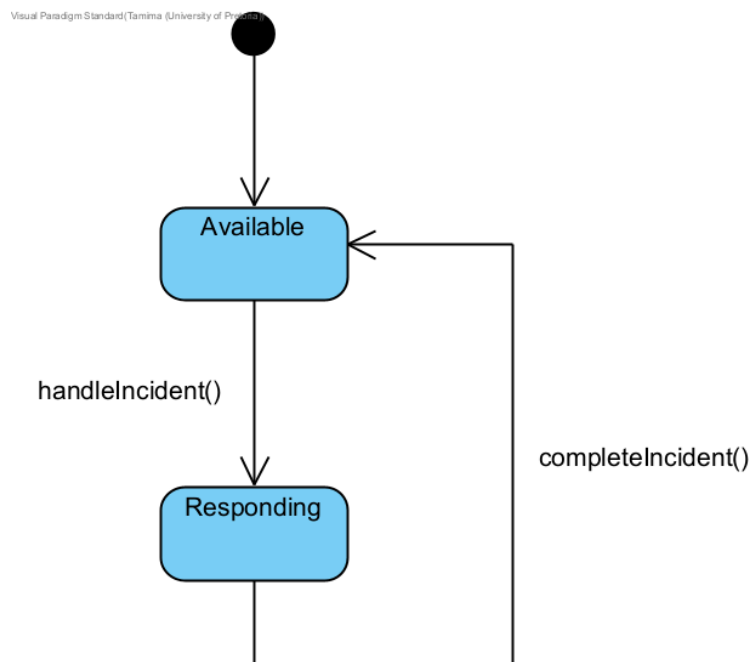
For ownership, the root **ResponseTeam** owns its nested components and deleting the root cleans up the hierarchy. Each Responder owns its current state and deletes the old state when changing between **Available** and **Responding**. Virtual destructors allow objects to be safely destroyed through base-class pointers. Decorated objects are also cleaned up carefully to prevent memory leaks and double deletion.

TASK 4: Diagrams

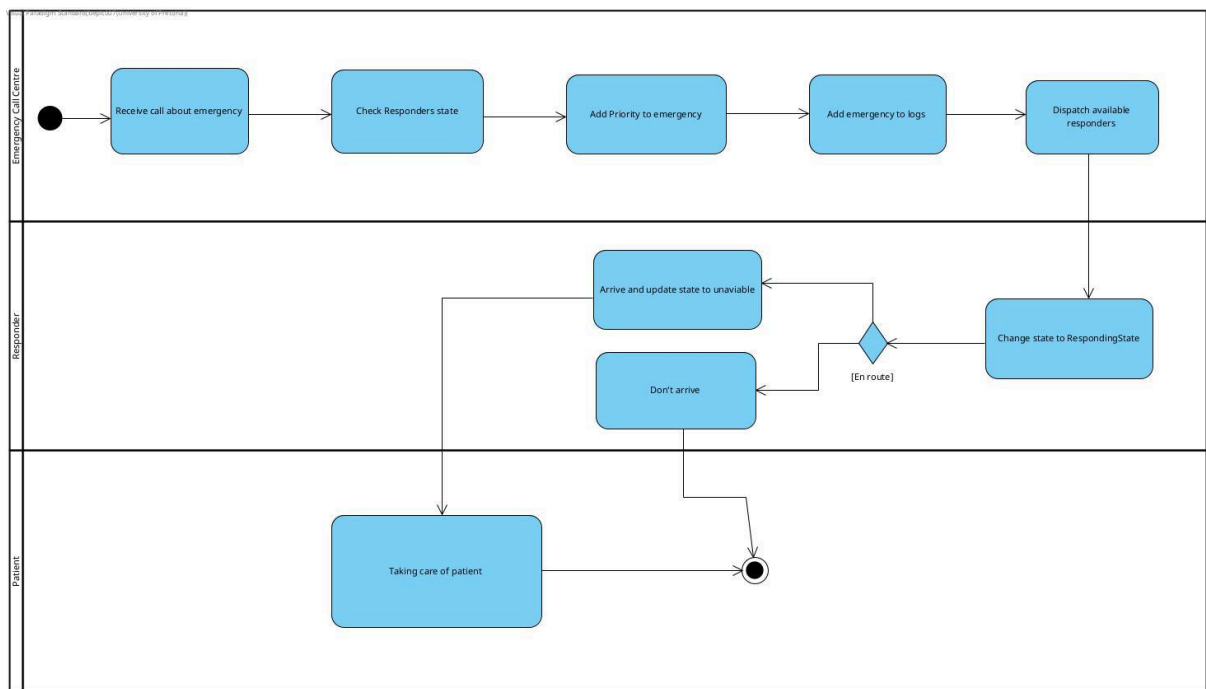
UML Object Diagram – Emergency Response Hierarchy



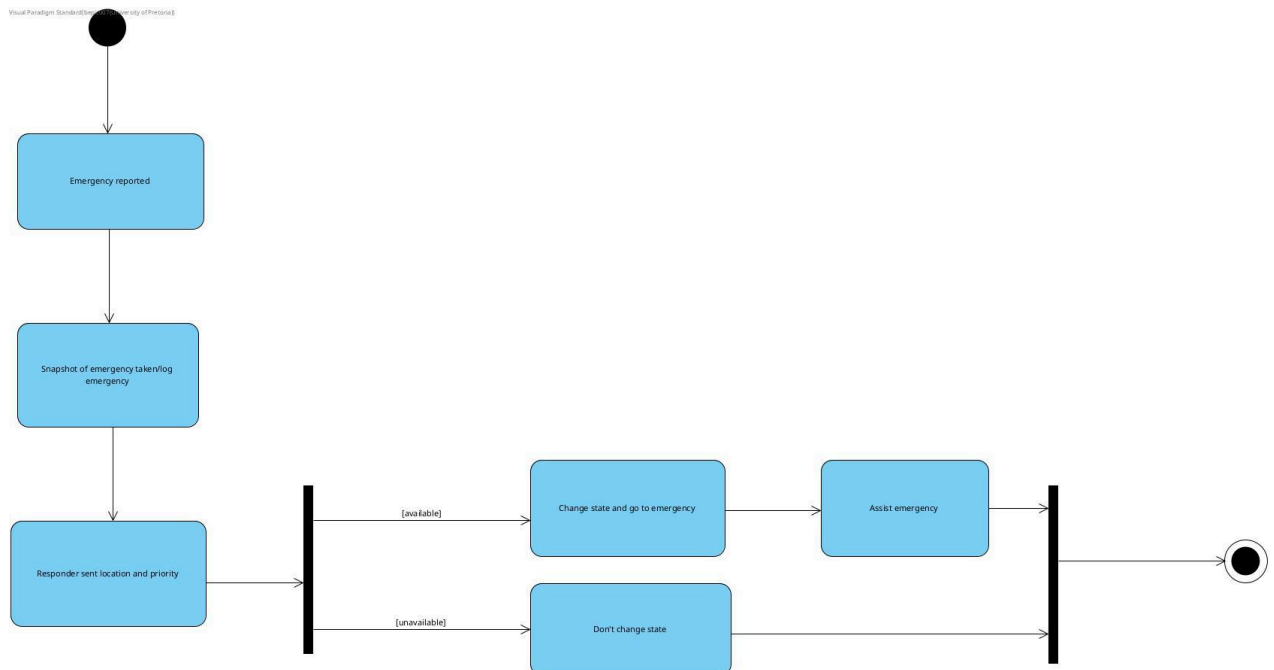
UML State Diagram – Responder Lifecycle



Activity Diagram 1 - SwimLanes

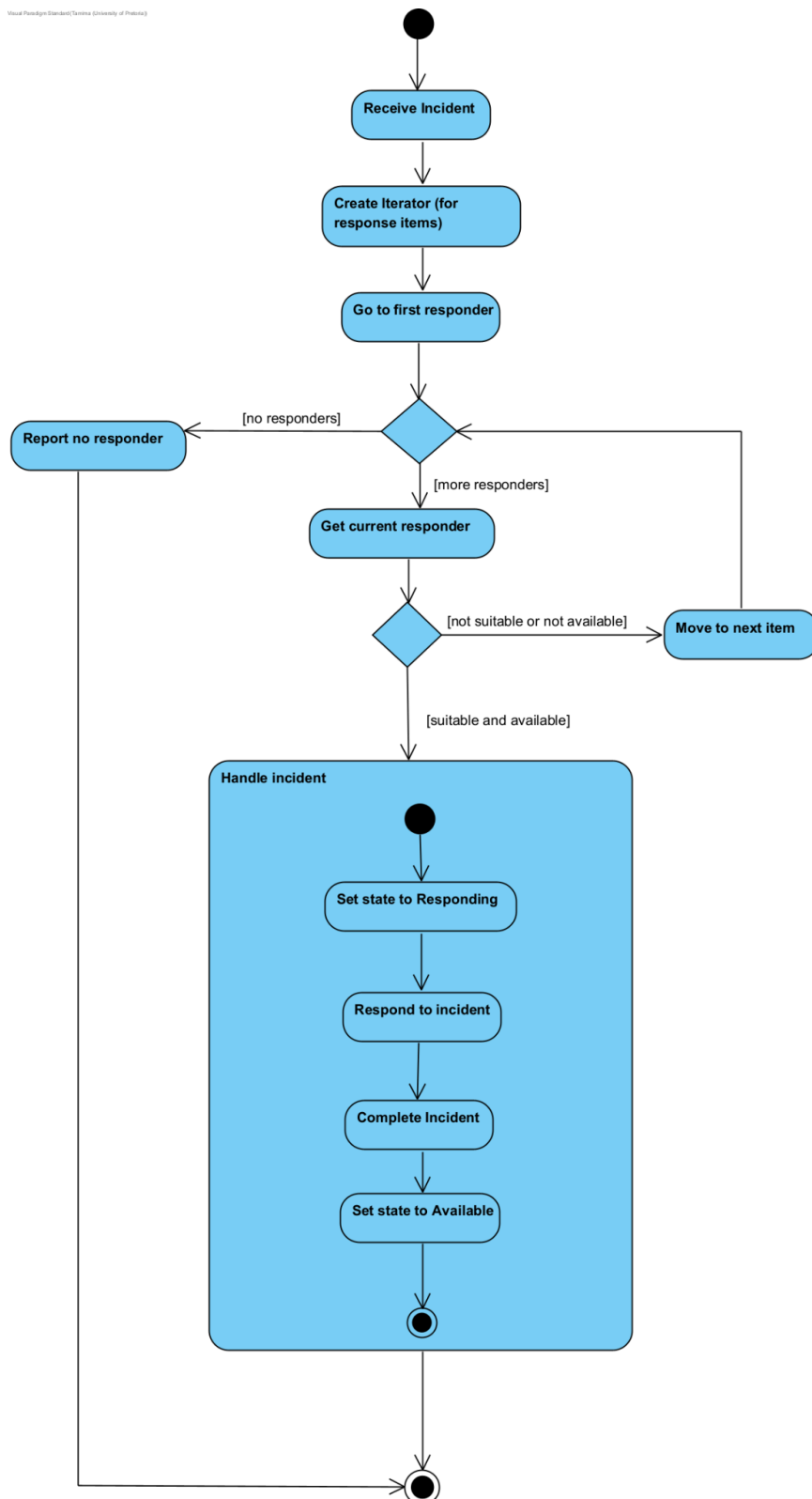


Activity Diagram 2 - merging and parallel processing



Activity Diagram 3 - composite activity and traversal

Visual Paradigm Standard/Terrace (University of Pretoria)



TASK 5:

Breakpoint and incident inspection

- Breakpoint
- Stepping - next
- Inspection - print

GDB was used to debug EmergencyResponseSystem::dispatchIncident(). A breakpoint was placed in the function and the incident details were inspected. The incident was Medical and located in "Pretoria CBD".

```
ead_db.so.1 .
----- TASKFORGE EMERGENCY RESPONSE SYSTEM -----

----- SCENARIO 1: MEDICAL EMERGENCY -----
Incident reported: Person injured in road accident
Location: Pretoria CBD

Dispatching responder...

Breakpoint 1, EmergencyResponseSystem::dispatchIncident (
  this=0x7fffffffdd60) at EmergencyResponseSystem.cpp:21
21 void EmergencyResponseSystem::dispatchIncident() {
(gdb) print currentIncident->description
$1 = "Person injured in road accident"
(gdb) print currentIncident->address
$2 = "Pretoria CBD"
(gdb) print currentIncident->type
$3 = Medical
(gdb) □
```

The program was then stepped through using next. When a responder was reached, its type was Medical and isAvailable() returned true.

```
(gdb) next
37     if (responder != nullptr) {
(gdb) next
42     movement->next();
(gdb) next
34     while (!movement->isDone()) {
(gdb) next
35         ResponseComponent* component = movement->currentItem();
(gdb) next
36         Responder* responder = dynamic_cast<Responder*>(component);
(gdb) next
37         if (responder != nullptr) {
(gdb) next
42         movement->next();
(gdb) next
34         while (!movement->isDone()) {
(gdb) next
35             ResponseComponent* component = movement->currentItem();
(gdb) next
36             Responder* responder = dynamic_cast<Responder*>(component);
(gdb) next
37             if (responder != nullptr) {
(gdb) next
38                 if (responder->getType() == currentIncident->type && responder->isAvailable()) {
(gdb) print responder
$5 = (Responder *) 0x55555572780
(gdb) print responder->getType()
$6 = Medical
(gdb) print responder->isAvailable()
$7 = true
(gdb) □
```

Valgrind output before fixing the memory leak

The leak happened because only the outer decorator was deleted, leaving the wrapped decorator and responder allocated. This was fixed by storing and deleting each object separately.

```
delete logs;
```

```
delete priority;
```

```
delete decoratedResponder;
```

```
==1075==
==1075== HEAP SUMMARY:
==1075==   in use at exit: 56 bytes in 3 blocks
==1075== total heap usage: 52 allocs, 49 frees, 76,048 bytes allocated
==1075==
==1075== 8 bytes in 1 blocks are indirectly lost in loss record 1 of 3
==1075==   at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1075==   by 0x10C64E: Responder::Responder(Type) (Responder.cpp:9)
==1075==   by 0x10DD40: main (main.cpp:65)
==1075==
==1075== 24 bytes in 1 blocks are indirectly lost in loss record 2 of 3
==1075==   at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1075==   by 0x10DD2A: main (main.cpp:65)
==1075==
==1075== 56 (24 direct, 32 indirect) bytes in 1 blocks are definitely lost in loss record 3 of 3
==1075==   at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1075==   by 0x10DD64: main (main.cpp:66)
==1075==
==1075== LEAK SUMMARY:
==1075==   definitely lost: 24 bytes in 1 blocks
==1075==   indirectly lost: 32 bytes in 2 blocks
==1075==   possibly lost: 0 bytes in 0 blocks
==1075==   still reachable: 0 bytes in 0 blocks
==1075==   suppressed: 0 bytes in 0 blocks
==1075==
==1075== For lists of detected and suppressed errors, rerun with: -s
==1075== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Valgrind output after fixing the memory leak

```
==1128==
==1128== HEAP SUMMARY:
==1128==   in use at exit: 0 bytes in 0 blocks
==1128== total heap usage: 52 allocs, 52 frees, 76,048 bytes allocated
==1128==
==1128== All heap blocks were freed -- no leaks are possible
==1128==
==1128== For lists of detected and suppressed errors, rerun with: -s
==1128== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@MidnightBlack:/mnt/c/Users/tamim/Desktop/COS 214 P4#
```

TASK 6: Workflow reflection

We used separate Git branches to keep the project organised and avoid conflicts while working on different tasks. The develop branch was mainly used for integrating and testing the source code, while the diagrams branch was used for creating and updating the UML diagrams. The task6-makefile-and-docker branch was used specifically for the Makefile, Dockerfile and related Task 6 work.

Once the different parts were completed and checked, they were merged into main, which contains the final source code, diagrams, Makefile, Dockerfile and other submission files. This workflow allowed team members to work independently on different parts of the practical while still combining everything into one final version.

Team Contribution

For the practical, Bianca completed Task 1, created two of the UML Activity Diagrams and set up the Google Doc and GitHub repository. She also created the initial Makefile and Dockerfile.

Christian completed Task 2, where he developed the emergency response scenario and implemented the main classes and files for the system.

Tamima completed Task 3 and created the main.cpp, Object Diagram, State Diagram and one Activity Diagram. She also completed Task 5 by doing the GDB debugging and Valgrind memory testing and worked on Task 6 by finalising and testing the Makefile and Dockerfile and creating the README.